

Relatório de Auditoria de Segurança — OBW Pools

Avaliação exaustiva de vulnerabilidades arquiteturais, controle de acesso, isolamento de dados e riscos de injeção.

Projeto: OBW Pools (WandPool)	Data da Auditoria: 28 de Agosto de 2026
Versão Avaliada: 1.0.0 (FastAPI + React 19)	Tipo de Auditoria: White-Box / Source Code Review
Classificação: Confidencial / Engenharia	Status Geral: Requer Remediação Imediata (P1)

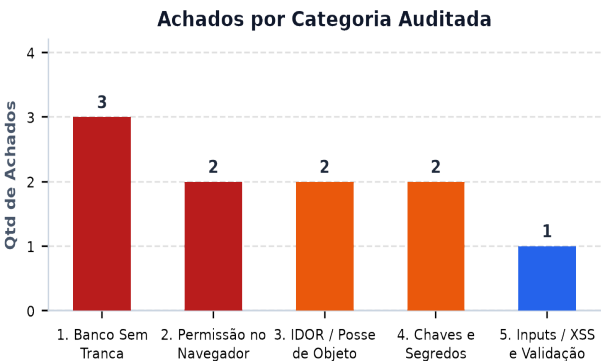
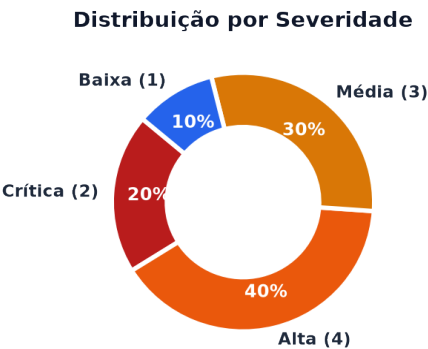
1. Detecção da Stack & Nota Metodológica

A auditoria realizou o rastreamento integral do repositório OBW Pools, identificando os componentes tecnológicos e mapeando as cinco categorias de segurança para o contexto técnico específico desta stack:

Camada	Tecnologia Detectada	Mapeamento Metodológico da Categoria de Segurança
Backend API	Python 3.12+ / FastAPI	Análise de rotas, dependências (Depends), middlewares e políticas CORS.
Banco / ORM	SQLite nativo (sqlite3)	BANCO SEM TRANCA: Ausência de tenant_id e filtros em queries SQL diretas.
Frontend Web	React 19, TypeScript, Vite 8	PERMISSÃO NO BROWSER: Verificação de gates em /portal sem espelhamento na API.
Mobile App	Capacitor 8.5 (Android / Camera / GPS)	Armazenamento local (localStorage) e segurança de dados de telemetria.
Deploy / CI	Cloudflare Pages (wrangler.toml)	CHAVES EXPOSTAS: Análise de .env.production e segredos no startup.

2. Resumo Executivo

Foram identificados **10 achados de segurança (2 Críticos, 4 Altos, 3 Médios e 1 Baixo)**. A causa raiz é a **ausência completa de autenticação no backend FastAPI**, combinada com uma falsa sensação de proteção decorrente do roteamento no cliente React (que apenas oculta visualmente o painel sem blindar os endpoints REST).



3. Análise de Postura: Pontos Fortes e Riscos Centrais

PONTOS FORTES (Defesas Ativas Verificadas)	RISCOS CENTRAIS (Vulnerabilidades Encontradas)
• Queries SQL 100% Parametrizadas: Em server/db.py, todas as operações usam marcadores ? (placeholders), neutralizando Injeção de SQL (SQLi). • Escape Nativo contra XSS no React 19: Ausência total de dangerouslySetInnerHTML, eval() ou innerHTML. • Headers HTTP de Proteção: Arquivo client/public/_headers configurado com X-Frame-Options: DENY e X-Content-Type-Options: nosniff. • Validação com Pydantic: Modelos de entrada bem definidos no FastAPI.	• API Pública sem Autenticação: 100% dos endpoints REST abertos sem exigência de JWT, Bearer Token ou Sessão. • Vazamento de Códigos de Portão (Gate Codes): O endpoint GET /api/pools entrega códigos de acesso físico a residências para qualquer chamador anônimo. • IDOR Generalizado: Rotas como PUT /api/pools/{id} e DELETE /api/technicians/{id} permitem sequestro ou destruição de dados. • CORS Permissivo: allow_origins=[*] com credenciais.

4. Tabela de Achados Detalhados por Categoria

Listagem exhaustiva de achados verificados no código-fonte real, mapeados linha por linha:

Sev.	Categoria	Arquivo e Linha(s)	Descrição e Impacto de Segurança
CRÍTICA	1. Banco Sem Tranca	server/db.py:574-586 server/main.py:221-224	Vazamento irrestrito de dados e códigos de portão de clientes: GET /api/pools executa SELECT * FROM pools sem autenticação ou filtro de tenant, expondo nomes, telefones, endereços e gate_code (códigos físicos de acesso a residências).
CRÍTICA	2. Permissão no Browser	client/src/App.tsx:28-30 server/main.py:77-315	Painel administrativo bloqueado só na UI do React: A alternância para a área operacional depende de isPortalPath(pathname). Todos os 14 endpoints de escrita (CRUD de técnicos, rotas e piscinas) na API aceitam chamadas diretas não autenticadas.
ALTA	3. IDOR / Posse	server/main.py:240-246 server/db.py:805-810	Alteração não autorizada de piscina e código de portão (IDOR): PUT /api/pools/{pool_id} atualiza qualquer registro no banco pelo ID do path sem checar autorização, permitindo sequestro cadastral ou adulteração de gate codes.
ALTA	3. IDOR / Posse	server/main.py:90-96 server/db.py:884-897	Exclusão arbitrária de técnicos e rotas (IDOR / Destrutivo): DELETE /api/technicians/{tech_id} remove o funcionário e apaga em cascata todas as rotas vinculadas a ele sem validação de papel admin.
ALTA	4. Chaves Expostas	server/db.py:193-349 client/src/lib/api.ts:663-810	Códigos reais de portão/acesso físico hardcoded: Instruções de portão (ex: 'Gate #4821', 'Código Portão *7720') estão fixadas no código-fonte Python e distribuídas nos fallbacks JavaScript do cliente.
ALTA	1. Banco Sem Tranca	server/main.py:47-53	CORS Wildcard com Credenciais Habilitadas: Configuração allow_origins=["*"] e allow_credentials=True permite que scripts de terceiros em qualquer aba do navegador leiam dados da API.
MÉDIA	1. Banco Sem Tranca	server/db.py:816-831 server/main.py:72-75	Exposição irrestrita da equipe de funcionários: GET /api/technicians lista nomes completos, telefones, e-mails e rotas ativas sem autenticação.
MÉDIA	2. Permissão no Browser	client/src/lib/leadGuard.ts:7-13 TurnstileWidget.tsx:51-73	Validação de Bot/Turnstile restrita ao frontend: O token Cloudflare Turnstile e o honeypot não são verificados via chamada server-side (siteverify), permitindo bypass por bots.
MÉDIA	4. Chaves Expostas	client/.env.production:1 server/main.py:34-57	Arquivo .env.production versionado no git e ausência de verificação de startup: Falta validação de variáveis de ambiente obrigatórias e segredos criptográficos no boot do FastAPI.
BAIXA	5. Inputs / XSS	client/src/components/ServiceChecklist.tsx:123	Interpolação de URLs sem validação de esquema de protocolo: Chamadas window.open constroem URLs de WhatsApp/Maps interpolando dados do banco sem sanitizar o esquema.

5. Plano de Remediação Priorizado

Prioridade	Ação Recomendada	Impacto e Prazo Sugerido
P1 (Crítica)	Implementar Autenticação Obrigatória no FastAPI: Adicionar middleware / dependência de autenticação (JWT / OAuth2 Bearer) em todas as rotas /api/*. Proteger imediatamente a leitura e escrita de piscinas e códigos de portão.	Elimina vazamento de dados físicos de clientes. <i>Prazo: Imediato (< 24h)</i>
P2 (Alta)	Adicionar Tenant Scoping & Validação de Posse (Anti-IDOR): Introduzir coluna tenant_id e user_id nas tabelas do SQLite. Validar se o usuário logado possui autorização para alterar a piscina ou rota especificada antes de executar o UPDATE ou DELETE.	Impede sequestro de contas e destruição de dados entre técnicos e empresas. <i>Prazo: 48h</i>
P3 (Média)	CORS Restritivo e Validação de Turnstile no Servidor: Restringir allow_origins para os domínios oficiais da empresa (ex: https://obwpools.com). Criar endpoint no FastAPI para validar o token do Cloudflare Turnstile via API antes de aceitar leads.	Bloqueia automações maliciosas e requisições cross-site. <i>Prazo: 1 semana</i>
P4 (Baixa)	Limpeza de Fallbacks com Dados Sensíveis e Sanitização de URLs: Remover senhas e códigos reais de portão dos arquivos client/src/lib/api.ts e server/db.py. Aplicar sanitizador de esquema em links de WhatsApp e mapas.	Higiene de código e eliminação de vestígios estáticos. <i>Prazo: Próximo sprint</i>

6. Issues Prontas para o GitHub (Copiar & Colar)

Textos integrais formatados em Markdown para abertura direta de issues no GitHub, contendo descrição técnica, evidência, impacto, sugestão de patch e checklist de aceite:

ISSUE 1 — Falta de Autenticação e Isolamento de Tenant na API de Piscinas

```
--- ISSUE 1 ---
## [Segurança] Implementar autenticação e isolamento de tenant na listagem e cadastro de piscinas

**Labels:** `security`, `critical`, `backend`, `auth`

### Descrição do Problema
O endpoint `GET /api/pools` executa uma consulta irrestrita `SELECT * FROM pools` no banco SQLite sem qualquer validação de autenticação (JWT/Session) ou isolamento de tenant/usuário.

### Por que é Explorável
Qualquer pessoa na internet pode enviar uma requisição GET para `/api/pools` e obter a lista completa de clientes cadastrados, telefones, e-mails, endereços e os códigos de acesso físico aos portões das residências (`gate_code`).

### Evidência
- **Arquivo:** `server/db.py:574-586` e `server/main.py:221-224`
  ```python
 @app.get("/api/pools", response_model=List[Dict[str, Any]])
 def list_pools():
 return get_all_pools()

 def get_all_pools():
 cursor.execute("SELECT * FROM pools ORDER BY name ASC")
 return cursor.fetchall()
 ...

Impacto
Vazamento crítico de dados pessoais (PII) e comprometimento da segurança física dos clientes (invasão de domicílio facilitada por códigos de portão expostos).

Sugestão de Correção
1. Adicionar dependência de autenticação `Depends(get_current_user)` nas rotas de pools.
2. Adicionar coluna `tenant_id` ou `owner_id` na tabela `pools` e filtrar queries: `WHERE tenant_id = ?`.
3. Ofuscar ou exigir permissão especial para visualização do campo `gate_code`.

Critérios de Aceite
- [] `GET /api/pools` sem header `Authorization: Bearer <token>` retorna HTTP 401 Unauthorized.
- [] Usuário logado enxerga apenas as piscinas associadas ao seu `tenant_id`.
- [] Campo `gate_code` não é retornado em listagens gerais.
--- FIM ISSUE 1 ---
```

## ISSUE 2 — Controle de Acesso Restrito ao Frontend (Bypass de Pannel Administrativo)

```
--- ISSUE 2 ---
[Segurança] Proteger endpoints de mutação no backend (Bypass de controle de acesso no cliente)

Labels: `security`, `critical`, `backend`, `rbac`

Descrição do Problema
O controle de acesso à área restrita de técnicos e administração é executado unicamente no cliente React através da verificação `isPortalPath(pathname)`. Os endpoints de escrita no servidor FastAPI não exigem autenticação nem validam privilégios de administrador.

Por que é Explorável
Um atacante pode ignorar completamente a interface do React e disparar requisições HTTP diretas (via curl/Postman) para endpoints como `POST /api/technicians`, `PUT /api/routes/{id}` ou `DELETE /api/technicians/{id}` e manipular toda a infraestrutura operacional da empresa.

Evidência
- **Frontend:** `client/src/App.tsx:28-30` | **Backend:** `server/main.py:77-157`
  ```python
  @app.post("/api/technicians")
  def create_technician(payload: Dict[str, Any]):
      return save_technician(payload)
  ```

Impacto
Manipulação não autorizada do quadro de funcionários, criação de rotas falsas e desvio operacional.

Sugestão de Correção
1. Implementar RBAC (Role-Based Access Control) no FastAPI com roles `admin`, `technician` e `client`.
2. Proteger rotas de mutação de equipe e rotas com verificação de role `admin`.

Critérios de Aceite
- [] Chamadas POST/PUT/DELETE em `/api/technicians` e `/api/routes` exigem autenticação.
- [] Requisições com role não autorizada retornam HTTP 403 Forbidden.
--- FIM ISSUE 2 ---
```

## ISSUE 3 — IDOR na Atualização e Exclusão de Piscinas, Técnicos e Paradas

```
--- ISSUE 3 ---
[Segurança] Corrigir IDOR em rotas de atualização e exclusão (Piscinas, Técnicos e Rotas)

Labels: `security`, `high`, `backend`, `idor`

Descrição do Problema
Endpoints parametrizados por ID (`PUT /api/pools/{pool_id}`, `DELETE /api/technicians/{tech_id}`, `DELETE /api/routes/stops/{stop_id}`) confiam cegamente no identificador fornecido na URL sem verificar se o recurso pertence ao usuário ou organização que fez a chamada.

Por que é Explorável
Basta alterar o ID na URL (ex: `PUT /api/pools/pool-1`) para modificar o endereço, telefone ou código de portão de outro cliente, ou apagar técnicos e suas agendas de atendimento.

Evidência
- **Arquivo:** `server/main.py:240-246`
  ```python
  @app.put("/api/pools/{pool_id}")
  def update_pool(pool_id: str, pool: Pool):
      pool_data = pool.dict()
      updated = update_pool_in_db(pool_id, pool_data)
  ```

Impacto
Sobrescrita indevida de cadastros de clientes, adulteração de coordenadas GPS de rotas e exclusão acidental ou maliciosa de paradas de serviço.

Sugestão de Correção
1. Antes de atualizar ou deletar, consultar se o recurso pertence ao `tenant_id` autenticado.
2. Utilizar UUIDs v4 não sequenciais e validar a posse do registro no banco antes da execução do `UPDATE`/`DELETE`.

Critérios de Aceite
- [] Tentativa de alterar recurso de outro tenant retorna HTTP 403 ou 404.
- [] Testes automatizados cobrindo tentativas de IDOR entre dois usuários distintos.
--- FIM ISSUE 3 ---
```

## ISSUE 4 — Códigos de Portão e Credenciais Físicas Hardcoded no Código e no Banco

```
--- ISSUE 4 ---
[Segurança] Remover códigos de acesso físico hardcoded e isolar fallbacks do cliente

Labels: `security`, `high`, `privacy`, `cleanup`

Descrição do Problema
Códigos reais de portão de clientes e condomínios no Texas (ex: `Gate #4821`, `Código Portão *7720`, `Keycard Guarita Les
te`) estão hardcoded no script de seed do banco de dados (`server/db.py`) e no arquivo de fallbacks offline do frontend (
`client/src/lib/api.ts`).

Por que é Explorável
Os dados do frontend em `client/src/lib/api.ts` são empacotados no bundle JavaScript distribuído para qualquer usuário qu
e acessar a Landing Page pública, expondo credenciais físicas de residências.

Evidência
- **Frontend:** `client/src/lib/api.ts:677, 707, 737, 767, 797` | **Backend:** `server/db.py:202, 236, 269, 301, 334`
 ``typescript
 gate_code: '#8842',
 gate_code: 'Código: 1984',
 gate_code: 'Portão Lateral Destravado',
 ...

Impacto
Exposição pública de instruções de entrada em residências particulares de clientes.

Sugestão de Correção
1. Substituir todos os fallbacks do frontend por dados fictícios (ex: `gate_code: 'Instruções fornecidas na chegada')`.
2. Armazenar códigos de portão no backend apenas sob criptografia em repouso e transmissão autenticada.

Critérios de Aceite
- [] Nenhum código de portão real ou sensível presente no bundle gerado em `client/dist`.
- [] Fallbacks estáticos do React não contêm telefones ou endereços de clientes reais.
--- FIM ISSUE 4 ---
```

## ISSUE 5 — Configuração Insegura de CORS e Validação de Startup no FastAPI

```
--- ISSUE 5 ---
[Segurança] Restringir política de CORS e adicionar validação de configuração no startup

Labels: `security`, `medium`, `configuration`, `cors`

Descrição do Problema
O middleware de CORS está configurado com `allow_origins=["*"]` e `allow_credentials=True`. Além disso, o servidor não po
ssui validação de variáveis de ambiente no startup para garantir o carregamento de segredos essenciais.

Por que é Explorável
Qualquer aplicação web rodando no navegador de um técnico pode efetuar requisições autenticadas para a API local ou remot
a e ler respostas confidenciais.

Evidência
- **Arquivo:** `server/main.py:47-53`
 ``python
 app.add_middleware(
 CORSMiddleware,
 allow_origins=["*"],
 allow_credentials=True,
 allow_methods=["*"],
 allow_headers=["*"],
)
 ...

Impacto
Vulnerabilidade a ataques de Cross-Origin Data Leakage.

Sugestão de Correção
1. Substituir `allow_origins=["*"]` por lista explícita carregada de variável de ambiente `CORS_ORIGINS`.
2. Adicionar validação no lifespam do FastAPI para falhar o boot se variáveis críticas estiverem ausentes.

Critérios de Aceite
- [] Requisições com `Origin: https://malicious-site.com` são bloqueadas pelo CORS.
- [] Variáveis de ambiente validadas via Pydantic `BaseSettings`.
--- FIM ISSUE 5 ---
```